Sprint Ceremony Document: Sprint 1

Project Name: ALT-F4
Sprint Window: October 18 – October 24, 2026
Overall Completion: 47/168 * 100 = **27.98%**
Sprint Completion: 24/24 * 100 = **100%**

Sprint Planning & Progress Update

Completed Tasks

1. User Registration & Account Security

- Comprehensive Validation: Verifies that all names, emails, student/staff affiliations, and passwords follow strict clinic rules before accounts can be created.
- Duplicate Protection: Automatically blocks duplicate email registrations.
- Secure Storage: Passwords are never stored as plain text—they are securely hashed before reaching the database.
- Clean Data Formatting: Automatically trims whitespace and standardizes input to prevent login formatting issues.

2. Login, Logout & Session Management

- Dynamic User Sessions: Connects directly to the database so real patient and staff profiles appear immediately upon login (no fake or hardcoded mock data).
- Refresh-Proof Login: Keeps users signed in even if they refresh the page or restart their browser.
- True Server Logout: Logging out invalidates the session token on the server and completely cleans local data, preventing unauthorized re-use.
- Dashboard Sign-Out Controls: Added simple, responsive "Sign Out" buttons to both patient and staff headers.

3. Role-Based Access Control & Page Protection

- Clear Role Separation: Clearly distinguished university roles (Student, Faculty, Staff) from clinic permissions (Patient, Clinic Staff, Admin).
- Strict Page Guards:
    - Patients are automatically blocked from entering Staff dashboards.
    - Staff members are directed straight to staff tools.
    - Logged-in users are prevented from seeing the Login/Register screens again.
- Inactive Account Handling: Deactivated accounts are instantly blocked from accessing the system.
- Server-Side API Security: The server checks every sensitive request and rejects anyone attempting actions outside their permissions.

4. User Interfaces & Forms

- Modern Auth Screens: Built responsive Login and Registration pages matching Figma mockups.
- Shared UI Components: Created reusable building blocks (Clinic Branding header, Auth Card container, and Affiliation dropdown).
- Human-Readable Errors: Replaced confusing server code errors with clear messages (e.g., "Cannot reach server" or "Invalid email or password").
- Smooth Navigation: Added direct links between login and registration, and remembers which page a user tried to visit before logging in.

5. Dashboard Shells & Layouts

- Patient Portal Shell: Added placeholders for active queue tickets, current ticket being served, and clinic schedules.
- Staff Console Shell: Created layout sections for active consultations, waiting lines, and patient statistics.
- Responsive Layouts: Designed to adapt cleanly across mobile and desktop displays.

6. Real-Time Queue Technology Upgrade (SSE)

- Lightweight Push Technology: Replaced heavy, complex WebSocket connections with HTTP-native Server-Sent Events (SSE).
- Live Notifications: Allows clinic displays and dashboards to receive instantaneous ticket calls and waiting line updates without connection drops.

7. Testing & Quality Assurance

- Full Test Matrix: Designed and carried out end-to-end and component tests for authentication, route access, and roles.
- Early Defect Detection: Caught and fixed integration mismatches early, documenting incomplete dashboard items to roll into Sprint 3.

Planned Tasks for Next Sprint (Sprint 2):

Sprint 3

Task ID	Task Title	Description
CQ-S3-3.1	Implement clinic slot APIs and validation	Implement slot listing and create/update/delete endpoints with date, time, capacity, overlap and active-appointment rules from the API specification.
CQ-S3-3.2	Build staff schedule-management interface	Create the staff interface for listing, adding, editing and controlling clinic slots.
CQ-S3-3.3	Build patient availability and booking interface	Build the patient view for browsing available slots, selecting an appointment time, confirming booking and reviewing upcoming appointments.
CQ-S3-3.4	Implement appointment creation and management APIs	Implement appointment create/list/detail and cancel/reschedule behavior based on confirmed user stories; keep capacity consistent.
CQ-S3-3.5	Integrate booking UI, appointment APIs and staff slots	Connect patient booking screens to appointment endpoints and verify patient booking actions stay consistent with staff schedule changes.
CQ-S3-3.6	Test schedule and appointment rules	Test validation, full/unbookable slots, overlap/capacity constraints, duplicate submissions and cancel/reschedule paths.

Sprint Retrospective

What Went Well:

- End-to-End Security & Input Normalization (CQ-S2-2.1 & 2.2)
- Frontend Route Protection & Session Hydration (CQ-S2-2.2 & 2.5)
- UI/UX Form Quality & Component Reusability (CQ-S2-2.3)
- Real-Time Architecture Upgrade (SSE Migration)
- Structured Quality Assurance (CQ-S2-2.4 & 2.6)

What Could Be Improved:

- Client-Server Error Contract Mismatches:
    - Challenge: NestJS custom exception filters returned { error: string }, while frontend helpers originally checked for { message: string }, leading to generic error fallbacks on forms.
    - Resolution: Standardized the frontend error parsing utility to extract both formats seamlessly.
- Prisma Schema Generation Overhead:
    - Challenge: Schema modifications and fresh checkouts occasionally triggered TypeScript compilation errors until npx prisma generate was explicitly re-run.
    - Resolution: Embedded Prisma generation into the core build and setup verification scripts.

What Needs to Be Done:

- Implement clinic slot APIs and validation
- Build staff schedule-management interface
- Build patient availability and booking interface
- Implement appointment creation and management APIs
- Integrate booking UI, appointment APIs and staff slots
- Test schedule and appointment rules

Individual Reports

Sprint: CQ-S2-2.1

Member: John Albert D. Masangkay / Matthew Ryan R. Sabino
Role: Technical Lead / Project Manager
Task: Implement registration validation to ensure that user account creation follows the required validation and security rules.

Done:

- Validated full name, email, password, and affiliation during registration.
- Rejected duplicate email addresses to prevent duplicate accounts.
- Normalized user inputs before validation and storage.
- Hashed passwords server-side using Argon2 before saving them to the database.
- Ensured that only valid registration data is accepted for account creation

Sprint: CQ-S2-2.2

Member: John Albert D. Masangkay / Matthew Ryan R. Sabino
Role: Technical Lead / Project Manager
Task: Add server-side role checks to protected endpoints by implementing reusable authorization checks for role-specific API operations and rejecting unauthorized requests.

Done:

- Implemented reusable server-side authorization checks for protected API endpoints.
- Restricted role-specific API operations based on user permissions.
- Prevented unauthorized requests from accessing restricted operations.
- Improved API security by enforcing access control on the server side.

Sprint: CQ-S2-2.3

Member: Jereign Lim / Matthew Ryan Sabino
Role: UI/UX Designer / Project Manager
Task: Build registration and login screens from the supplied mockups and connect them to the authentication APIs.

Done:

- Built the Login and Registration screens in Angular, based on the mockups.
- Added form rules for required fields, email format, minimum password length and matching passwords.
- Added error messages under each field, a loading state on the button, and readable server and network error messages.

- Built one Auth layout and 3 reusable components (AuthBrand, AuthCard, AffiliationSelector) shared by both screens.
- Connected the forms to the existing auth service and sent each user to their home page by role.

Sprint: CQ-S2-2.4

Member: John Alec L. Resos / John Raphael Reganit
Role: Tester
Task: Build patient and clinic-staff dashboard shells
Done:

- Implemented a test matrix
- Updated the app.route.ts file to refine the application's initial routing
- Modified the login and register pages look
- Created initial dashboard templates for patient & staff
- Added navigation and section tracking to the patient dashboard, and added a profile summary and upcoming events placeholder to the staff dashboard.

Sprint: CQ-S2-2.5

Member: Matthew Ryan R. Sabino / John Albert D. Masangkay / Jereign Lim
Role: Project Manager / Tech Lead / UI/UX
Task: Integrate authentication and user profile state with protected dashboards by connecting login results to dynamic application state, verifying route and API access boundaries, and implementing secure server-side logout with safe redirects.
Done:

- Connected login and profile retrieval to reactive application state using Angular signals, replacing hardcoded mock profiles with dynamic database records.
- Implemented automatic session hydration on application startup to persist authentication state across page reloads.
- Built a functional HTTP interceptor to automatically attach JWT Bearer tokens to outgoing API calls and handle unauthorized session expiries.
- Tested and verified route guards and direct API access to ensure patients and staff are restricted to their authorized views.
- Implemented complete server-side token revocation on logout to blacklist tokens in the database and clear all client credentials.
- Integrated functional sign-out controls and safe redirect flows across both Patient and Staff dashboard views.

Sprint: CQ-S2-2.6

Member: John Alec L. Resos
Role: Tester
Task: Test authentication, roles and dashboard integration

Done:

- Conducted component testing in accordance with the test matrix.